

# CodeAlpha Python Programming Internship

## Task 1 Solution: Hangman Game Documentation & Source Code

### 1. Project Objective & Scope

The objective of this task is to create a simple, text-based Hangman game where the user guesses a hidden word letter by letter. In accordance with the project guidelines, the implementation adheres to the following specific constraints:

- Uses a small predefined list of 5 words (no external API or file dependency required).
- Limits the number of permitted incorrect guesses exactly to 6.
- Utilizes basic text-based console input and output for universal terminal support.

**Key Concepts Demonstrated:** Random word selection (random module), control flow with while loops and if-else conditions, string manipulation, and state tracking via lists.

### 2. Python Source Code

```
import random

def play_hangman():
    # 1. Predefined list of 5 words as specified in the scope
    words_list = ["python", "codealpha", "developer", "software", "internship"]

    # Randomly select a word from the list
    secret_word = random.choice(words_list)

    # Track correctly guessed letters and incorrect guesses
    guessed_letters = set()
    incorrect_guesses = 0
    max_incorrect_guesses = 6

    print("=== Welcome to CodeAlpha Hangman Game! ===")
    print(f"The secret word has {len(secret_word)} letters.")
    print(f"You can make up to {max_incorrect_guesses} incorrect guesses.")

    # Game loop
```

```

while incorrect_guesses < max_incorrect_guesses:
    # Display current word status (e.g., p _ t h _ n)
    display_word = [letter if letter in guessed_letters else "_" for letter
in secret_word]
    print("Word to guess: " + " ".join(display_word))

    # Check if the user has successfully guessed the whole word
    if "_" not in display_word:
        print(f"
🎉 Congratulations! You guessed the word correctly: '{secret_word}')"
        break

    # Get player input
    guess = input("Guess a letter: ").strip().lower()

    # Input validation
    if len(guess) != 1 not guess.isalpha():
        print("⚠️ Invalid input. Please enter exactly one alphabetic letter.
")
        continue

    if guess in guessed_letters:
        print(f"⚠️ You already guessed the letter '{guess}'. Try another
one.
")
        continue

    # Process the guess
    guessed_letters.add(guess)

    if guess in secret_word:
        print(f"✅ Good job! '{guess}' is in the word.
")
    else:
        incorrect_guesses += 1
        remaining = max_incorrect_guesses - incorrect_guesses
        print(f"❌ Incorrect guess! '{guess}' is not in the word.")
        print(f"Remaining incorrect guesses allowed: {remaining}
")

    # Game Over check
    if incorrect_guesses == max_incorrect_guesses:
        print("💣 Game Over! You have run out of guesses.")
        print(f"The secret word was: '{secret_word}')"

```

```
if __name__ == "__main__":  
    play_hangman()
```

### 3. Code Walkthrough & Implementation Logic

- **Word Selection:** The `random.choice()` method guarantees an unbiased selection from the predefined 5-word array.
- **State Management:** A Python set handles unique tracking of user inputs, eliminating redundant checks and keeping guess verification at  $O(1)$  performance.
- **Visual Rendering:** List comprehensions dynamically generate string patterns substituting unguessed letters with blank underscores (`_`) in real-time.